44

BÖLÜM: KUBERNETES OPERATORS – KENDI AKILLI ROBOTUNU KODLA | MODULE: KUBERNETES OPERATORS – BUILD YOUR OWN ROBOT SRE

CRD (Custom Resource Definition) ve Reconciliation Döngüsü

CRDs and the Infinite Reconciliation Loop

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kubernetes doğduğunda sadece 'Pod', 'Service' gibi birkaç kelime bilirdi. CRD (Custom Resource Definition), Kubernetes'e yepyeni kelimeler öğretmektir! 'Artık sen `SpringBootApp` veya `CoffeeMachine` kelimesini de bileceksin' deriz. Reconciliation (Mutabakat) Döngüsü ise robotun 'İstenen kahve hazırlandı mı? Eksik var mı?' diye etrafı sonsuz bir merakla kontrol etmesidir.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

When Kubernetes was born, it only understood basic words like 'Pod' and 'Service'. A CRD (Custom Resource Definition) teaches Kubernetes brand-new nouns! You say: 'From now on, you understand what a `SpringBootApp` is.' The Reconciliation Loop is the robot's infinite inner monologue: 'I see 1 desired `SpringBootApp`; do 3 matching pods exist? If not, make it so!'

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

1) CRD: K8s API Server'a OpenAPI v3 şemasıyla yeni bir REST uç noktası (`/apis/apps.eli5.io/v1alpha1/...`) ekler. 2) Custom Resource (CR): Bu şablon kullanılarak oluşturulan gerçek nesnedir. 3) Reconciliation Loop: Controller sürekli olarak K8s API'sindeki Watch kuyruğunu dinler. `Reconcile(Request)` fonksiyonu çağrıldığında: Mevcut kaynakları sorgular, aradaki farkı hesaplar ve K8s API çağrılarıyla sistemi olması gereken duruma eşitler.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

1) CRDs register custom REST endpoints into the API Server with OpenAPI v3 schema validation. 2) Custom Resources (CR) are instances conforming to that schema. 3) The Reconciliation Loop triggers on Watch event queues. The `Reconcile(Request)` method retrieves the observed state, calculates the delta against desired state, and issues idempotent API calls to eliminate drift.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# springbootapp-crd.yaml (Yeni Nesne Tanımı)
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: springbootapps.apps.eli5.io
spec:
  group: apps.eli5.io
  versions:
    - name: v1alpha1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                image: { type: string }
                replicas: { type: integer, minimum: 1, maximum: 10 }
                port: { type: integer, default: 8080 }
  scope: Namespaced
  names:
    plural: springbootapps
    singular: springbootapp
    kind: SpringBootApp
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Reconciliation döngüsündeki kodunuz MUTLAKA 'Idempotent' (Tekrarlanabilir) olmalıdır! Döngü 1 kez de çalışsa, peş peşe 100 kez de çalışsa aynı sonucu üretmeli, her seferinde sıfırdan yeni kaynak yaratmamalıdır.

🇬🇧 WATCH OUT

Reconciliation logic MUST be strictly idempotent! Whether executed once or a thousand times, it must evaluate current state and produce the exact same outcome.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- CRD'ler Kubernetes API'sini kendi alan adınıza özel nesnelerle genişletir.
- Reconciliation döngüsü 'İstenen Durum' ile 'Mevcut Durum'u daima eşitler.
- OpenAPI v3 doğrulaması hatalı YAML girişlerini anında reddeder.

- CRDs extend the core Kubernetes API with domain-tailored schema endpoints.
- The reconciliation loop relentlessly drives actual state toward desired state.
- Built-in OpenAPI v3 validation rejects malformed YAML payloads at the API door.